

Имитационное моделирование

Лабораторная работа №4. Агентная эпидемиологическая модель SIR

Исаева Зарина Исмаилбековна

Содержание

Содержание

1. Цель работы
2. Задание
3. Теоретическое введение
4. Выполнение лабораторной работы
5. Выводы
6. Список литературы

Список иллюстраций

1. Среда Julia и структура проекта
2. Модуль агентной модели и сценарий оптимизации в редакторе
3. Базовая динамика SIR и параметрические варианты
4. Порог эпидемии и гетерогенность городов
5. Миграция и карантинные меры
6. Парето-фронт многокритериального поиска
7. Сводный анализ сохранённых данных
8. Выполненные Jupyter notebooks и автоматические проверки

1. Цель работы

Цель работы — реализовать пространственную стохастическую модель SIR в агентном подходе, исследовать влияние заразности, неоднородности городов, миграции и карантина, а также подобрать меры управления, минимизирующие смертность при ограничении пика заболеваемости.

2. Задание

Согласно главе 4 практикума [1] требуется:

1. создать воспроизводимый проект Julia и установить Agents.jl;
2. реализовать `src/sir_model.jl` с состояниями S, I и R, миграцией, выявлением, выздоровлением, смертью и повторным заражением;
3. выполнить пять основных сценариев: базовый опыт, сканирование β , миграцию, многокритериальную оптимизацию и сводную визуализацию;
4. определить R_0 , наблюдаемый порог эпидемии и влияние неоднородных β ;
5. добавить закрытие города при превышении порога заболеваемости;
6. найти параметры с пиком не выше 30% и минимальной смертностью;
7. создать параметрический вариант и для каждого из восьми литературных источников получить чистый Jupyter, выполненный IPYNB, QMD и HTML;
8. интегрировать в отчёт полные листинги, результаты и библиографию.

3. Теоретическое введение

3.1 Компартиментальная модель SIR

Модель Кермака—Маккендрика разделяет население на восприимчивых S, инфицированных I и выздоровевших R [2]. В детерминированной форме динамика задаётся системой

$$\frac{dS}{dt} = -\beta \frac{SI}{N}, \frac{dI}{dt} = \beta \frac{SI}{N} - \gamma I, \frac{dR}{dt} = \gamma I.$$

Для средней продолжительности инфекции T_{inf} используется $\gamma = 1/T_{inf}$, поэтому базовое репродуктивное число равно $R_0 = \beta T_{inf}$. При параметрах пособия $\beta = 0,5$ и $T_{inf} = 14$ дней получаем $R_0 = 7$.

3.2 Агентное представление

В агентной модели каждый человек имеет дискретное состояние, город и число дней после заражения. Локальные случайные контакты и индивидуальные события дают динамику, которую агрегированная система

ОДУ не описывает напрямую. Agents.jl предоставляет графовое пространство, планировщик и единый генератор случайных чисел [3]. Общие принципы построения и проверки агентных моделей изложены также в [4].

В модели невыявленный инфицированный создаёт пуассоновское число попыток заражения с интенсивностью β_{und} , а после дня выявления — с меньшей β_{det} . По окончании болезни агент выздоравливает либо удаляется с вероятностью `death_rate`. Матрица миграции является стохастической по строкам.

3.3 Воспроизводимость вычислений

Проект организован DrWatson [5]. Literate.jl превращает один комментированный Julia-файл в чистый сценарий, notebook и Quarto-документ, следуя принципу литературного программирования [6]. Все случайные серии закреплены `seed`, а сводная визуализация читает сохранённые CSV, не повторяя симуляцию.

4. Выполнение лабораторной работы

4.1 Окружение и структура

Проект содержит общий модуль, восемь литературных сценариев, тесты, исходные и производные документы, таблицы CSV/JLD2 и графики. Среда закреплена в `Project.toml`; команды полного цикла собраны в `Makefile`.

```
rhktrlwmd@Mac % julia --project=. -e 'using Pkg; Pkg.status()'
Project AgentSIRLab v4.0.0
Status `~/workspace/labs/lab04/project/Project.toml`
 [46ada45e] Agents v7.0.3
 [336ed68f] CSV v0.10.17
 [13f3f980] CairoMakie v0.15.14
 [a93c6f00] DataFrames v1.8.2
 [31c24e10] Distributions v0.25.131
 [634d3b9d] DrWatson v2.19.1
 [7073ff75] IJulia v1.34.4
 [033835bb] JLD2 v0.6.6
 [98b081ad] Literate v2.21.0
 [2913bbd] StatsBase v0.34.13
rhktrlwmd@Mac %
```

Рисунок 1: Список пакетов Julia

```
rhktrlwmd@Mac % find . -maxdepth 2 -type d | sort
.
./data
./markdown
./notebooks
./plots
./scripts
./src
./test
rhktrlwmd@Mac %
```

Рисунок 2: Структура каталогов лабораторной

name	=	"AgentSIRLab"
uuid	=	"9ecb6262-3c2e-4ae5-a381-694487bbc5f6"
authors	=	["rhktrlwmd"]
version	=	"4.0.0"
[deps]		
Agents	=	"46ada45e-f475-11e8-01d0-f70cc89e6671"
CSV	=	"336ed68f-0bac-5ca0-87d4-7b16caf5d00b"
CairoMakie	=	"13f3f980-e62b-5c42-98c6-ff1f3baf88f0"
DataFrames	=	"a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
Distributions	=	"31c24e10-a181-5473-b8eb-7969acd0382f"
DrWatson	=	"634d3b9d-ee7a-5ddf-bec9-22491ea816e1"
IJulia	=	"7073ff75-c697-5162-941a-fcdaad2a7d2a"
JLD2	=	"033835bb-8acc-5ee8-8aae-3f567f8a3819"
Literate	=	"98b081ad-f1c9-55d3-8b20-4c87d4299306"
Random	=	"9a3f8284-a2c9-5f02-9a11-845980a1fd5c"
Statistics	=	"10745b16-79ce-11e8-11f9-7d13ad32a3b2"
StatsBase	=	"2913bbd2-ae8a-5f71-8c99-4fb6c76f3a91"
[compat]		
Agents	=	"7"
CairoMakie	=	"0.15"
Distributions	=	"0.25"
JLD2	=	"0.6"
StatsBase	=	"0.34"
julia	=	"1.11"
[preferences.Makie]		
precompile_workload	=	false

```
[preferences.CairoMakie]
precompile_workload = false
```

Листинг 1 — полный файл project/Project.toml.

```
SOURCES = scripts/sir_run_basic.jl scripts/sir_run_basic_param.jl scripts/sir_scan_beta.jl
scripts/sir_city_heterogeneity.jl scripts/sir_migration_effect.jl scripts/sir_quarantine.jl scripts/sir_optimize_parameters.jl
scripts/sir_visualize_dynamics.jl

.PHONY: setup tangle run notebooks docs test verify all clean-results

setup:
    julia --project=. scripts/setup_environment.jl

tangle:
    julia --project=. scripts/tangle.jl $(SOURCES)

run:
    @for source in $(SOURCES); do julia --project=. $$source || exit $$?; done

notebooks:
    julia --project=. scripts/tangle.jl --execute $(SOURCES)

docs:
    @for source in $(SOURCES); do \
        name=$(basename $$source .jl); \
        (cd notebooks/$$name && quarto render $$name.ipynb --to html --embed-resources --no-execute --output $$name.html) || \
        exit $$?; \
    done
    mv notebooks/$$name/$$name.html markdown/$$name/$$name.html;

test:
    julia --project=. test/runtests.jl

verify:
    @clean_scripts=$(find scripts -mindepth 2 -maxdepth 2 -name '*.jl' | wc -l | tr -d ' '); \
    notebooks=$(find notebooks -mindepth 2 -maxdepth 2 -name '*.ipynb' | wc -l | tr -d ' '); \
    qmd=$(find markdown -mindepth 2 -maxdepth 2 -name '*.qmd' | wc -l | tr -d ' '); \
    html=$(find markdown -mindepth 2 -maxdepth 2 -name '*.html' | wc -l | tr -d ' '); \
    figures=$(find plots -type f -name '*.png' | wc -l | tr -d ' '); \
    printf 'clean Julia scripts: %s\nexecuted notebooks: %s\nQuarto documents: %s\nHTML documents: %s\nresult figures: %s\n' \
        "$$clean_scripts" "$$notebooks" "$$qmd" "$$html" "$$figures"; \
    test "$$clean_scripts" = 8; test "$$notebooks" = 8; test "$$qmd" = 16; test "$$html" = 8; test "$$figures" -ge 10

all: run notebooks docs test verify

clean-results:
    rm -rf data plots notebooks markdown
```

Листинг 2 — полный файл project/Makefile.

```
using Pkg
Pkg.activate(joinpath(@__DIR__,
Pkg.instantiate()
Pkg.precompile()
println("Среда AgentSIRLab готова: ", Base.active_project())
```

Листинг 3 — подготовка среды.

```

using
@quickactivate
using

execute_notebooks = "--execute" in ARGS
sources = filter(!="--execute"), ARGS

for source in sources
    name = splitext(basename(source))[1]
    Literate.script(source, scriptsdir(name); name=name, credit=false)
    Literate.notebook(source, projectdir("notebooks", name);
        name=name, execute=execute_notebooks, credit=false)
    Literate.markdown(source, projectdir("markdown", name);
        name=name, flavor=Literate.QuartoFlavor(), credit=false)
    Literate.markdown(source, projectdir("markdown", name);
        name=name * "-report", flavor=Literate.QuartoFlavor(), credit=false,
        postprocess=text -> replace(replace(text, "{julia}" => "`julia`"),
            r"^(#{1,4}) "m => heading -> "##" * heading))
end

```

Листинг 4 — генерация производных форматов Literate.jl.

4.2 Общий модуль агентной модели

Модуль проверяет размеры городских параметров, нормировку миграционной матрицы и вероятностные ограничения. Модель использует полный граф городов, но интенсивность переходов задаётся матрицей. Карантин динамически блокирует исходящую миграцию из города, где доля инфицированных достигла порога.

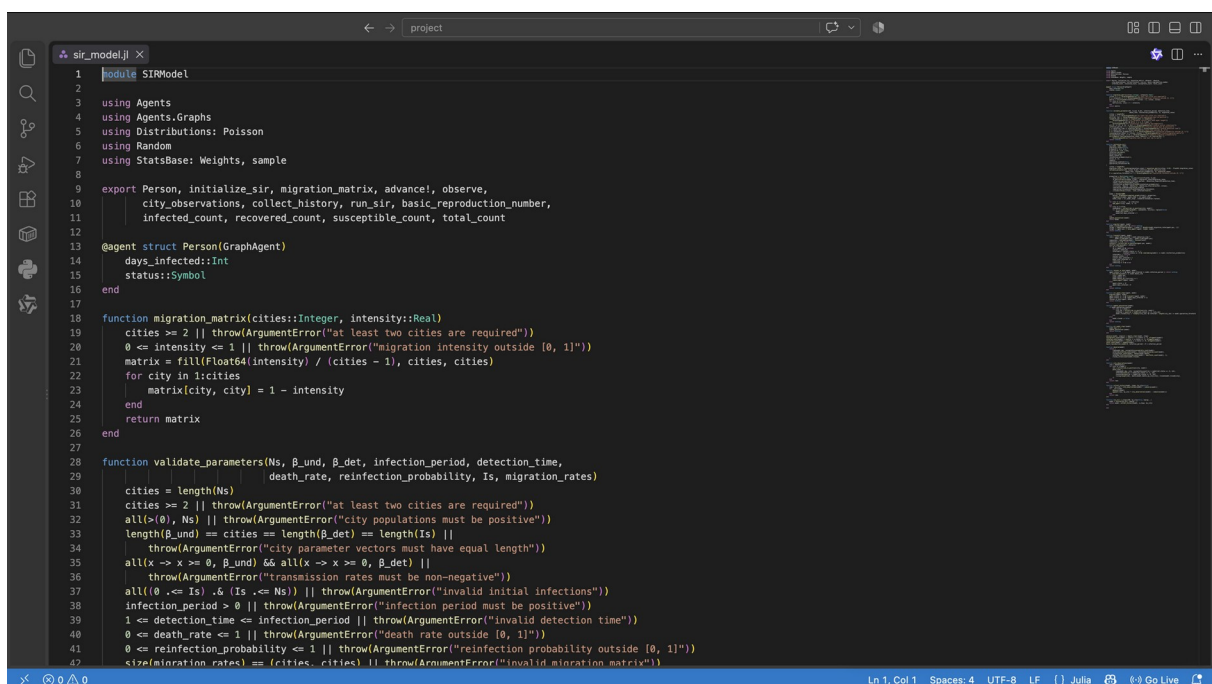


Рисунок 3: Модуль sir_model.jl в VS Code


```

module SIRModel

using Agents
using Agents.Graphs
using Distributions: Poisson
using Random
using StatsBase: sample

export Person, initialize_sir, migration_matrix, advance!, observe,
           city_observations, collect_history, run_sir, basic_reproduction_number,
           infected_count, recovered_count, susceptible_count, total_count

@agent struct Person{GraphAgent}
    days_infected::Int
    status::Symbol
end

function migration_matrix(cities::Integer, intensity::Real)
    cities >= 2 || throw(ArgumentError("at least two cities are required"))
    0 <= intensity <= 1 || throw(ArgumentError("migration intensity outside [0, 1]"))
    matrix = fill(Float64(intensity) / (cities - 1), cities, cities)
    for city in 1:cities
        matrix[city, city] = 1 - intensity
    end
    return matrix
end

function validate_parameters(Ns, β_und, β_det, infection_period, detection_time,
                             death_rate, reinfection_probability, Is, migration_rates)
    cities = length(Ns)
    cities >= 2 || throw(ArgumentError("at least two cities are required"))
    all(>(0), Ns) || throw(ArgumentError("city populations must be positive"))
    length(β_und) == cities == length(β_det) == length(Is) ||
        throw(ArgumentError("city parameter vectors must have equal length"))
    all(x -> x >= 0, β_und) && all(x -> x >= 0, β_det) ||
        throw(ArgumentError("transmission rates must be non-negative"))
    all((0 <= Is) .& (Is <= Ns)) || throw(ArgumentError("invalid initial infections"))
    infection_period > 0 || throw(ArgumentError("infection period must be positive"))
    1 <= detection_time <= infection_period || throw(ArgumentError("invalid detection time"))
    0 <= death_rate <= 1 || throw(ArgumentError("death rate outside [0, 1]"))
    0 <= reinfection_probability <= 1 || throw(ArgumentError("reinfection probability outside [0, 1]"))
    size(migration_rates) == (cities, cities) || throw(ArgumentError("invalid migration matrix"))
    all(migration_rates .>= 0) || throw(ArgumentError("negative migration probability"))
    all(isapprox.(vec(sum(migration_rates; dims=2)), 1.0; atol=1e-10)) ||
        throw(ArgumentError("migration matrix rows must sum to one"))
    return nothing
end

function initialize_sir(;
    Ns=[1000, 1000, 1000],
    migration_rates=nothing,
    β_und=[0.5, 0.5, 0.5],
    β_det=[0.05, 0.05, 0.05],
    infection_period=14,
    detection_time=7,
    death_rate=0.02,
    reinfection_probability=0.1,
    Is=[0, 0, 1],
    seed=42,
    quarantine_enabled=false,

```

```

)
quarantine_threshold=0.08,

cities = length(Ns)
migration_rates = isnothing(migration_rates) ? migration_matrix(cities, 0.04) : Float64.(migration_rates)
validate_parameters(Ns, β_und, β_det, infection_period, detection_time,
                    death_rate, reinfection_probability, Is, migration_rates)
0 <= quarantine_threshold <= 1 || throw(ArgumentError("quarantine threshold outside [0, 1]"))

properties = Dict{Symbol,Any}()
:Ns=>collect{Int, Ns}, :β_und=>collect{Float64, β_und},
:β_det=>collect{Float64, β_det}, :migration_rates=>migration_rates,
:infection_period=>Int(infection_period), :detection_time=>Int(detection_time),
:death_rate=>Float64(death_rate),
:reinfection_probability=>Float64(reinfection_probability),
:C=>cities, :day=>0, :deaths=>0, :deaths_by_city=>zeros{Int, cities},
:quarantine_enabled=>quarantine_enabled,
:quarantine_threshold=>Float64(quarantine_threshold),
:closed=>falses(cities), :ever_infected=>sum(Is),
)
model = StandardABM(
    Person, GraphSpace(complete_graph(cities)); properties,
    rng=Xoshiro(seed), agent_step! = sir_agent_step!,
    model_step! = sir_model_step!, scheduler=Schedulers.fastest,
)
for city in 1:cities, _ in 1:Ns[city]
    add_agent!(city, model, 0, :S)
end
for city in 1:cities
    candidates = collect(ids_in_position(city, model))
    for id in sample(abmrng(model), candidates, Is[city]; replace=false)
        model[id].status = :I
        model[id].days_infected = 1
    end
end
update_quarantine!(model)
return model
end

function migrate!(agent, model)
    target = sample(abmrng(model), 1:model.C, Weights(model.migration_rates[agent.pos, :]))
    target == agent.pos || move_agent!(agent, target, model)
    return nothing
end

function transmit!(agent, model)
    rate = agent.days_infected < model.detection_time ?
        model.β_und[agent.pos] : model.β_det[agent.pos]
    remaining = rand(abmrng(model), Poisson(rate))
    remaining == 0 && return nothing
    contacts = collect(ids_in_position(agent.pos, model))
    shuffle!(abmrng(model), contacts)
    for id in contacts
        id == agent.id && continue
        contact = model[id]
        infectable = contact.status == :S ||
            (contact.status == :R && rand(abmrng(model)) <= model.reinfection_probability)
        infectable && continue
        contact.status = :I
        contact.days_infected = 1
        model.ever_infected += 1
    end
end

```

```

        remaining == 0 && break
    end
    return nothing
end

function recover_or_die!(agent, model)
    agent.status == :I && agent.days_infected >= model.infection_period || return nothing
    if rand(abmrng(model)) <= model.death_rate
        city = agent.pos
        model.deaths += 1
        model.deaths_by_city[city] += 1
        remove_agent!(agent, model)
    else
        agent.status = :R
        agent.days_infected = 0
    end
    return nothing
end

function sir_agent_step!(agent, model)
    migrate!(agent, model)
    agent.status == :I && transmit!(agent, model)
    agent.status == :I && (agent.days_infected += 1)
    recover_or_die!(agent, model)
    return nothing
end

function update_quarantine!(model)
    if model.quarantine_enabled
        for city in 1:model.C
            city_ids = collect(ids_in_position(city, model))
            infected = count(id -> model[id].status == :I, city_ids)
            model.closed[city] = !isempty(city_ids) && infected / length(city_ids) >= model.quarantine_threshold
        end
    else
        model.closed .= false
    end
    return nothing
end

function sir_model_step!(model)
    model.day += 1
    update_quarantine!(model)
    return nothing
end

advance!(model, steps=1) = Agents.step!(model, steps)
susceptible_count(model) = count(a -> a.status == :S, allagents(model))
infected_count(model) = count(a -> a.status == :I, allagents(model))
recovered_count(model) = count(a -> a.status == :R, allagents(model))
total_count(model) = nagents(model)
basic_reproduction_number( $\beta$ , infection_period) =  $\beta$  * infection_period

function observe(model)
    return (
        time=model.day, susceptible=susceptible_count(model),
        infected=infected_count(model), recovered=recovered_count(model),
        living=total_count(model), deaths=model.deaths,
        infected_fraction=infected_count(model) / max(total_count(model), 1),
    )
end

```

```

                                closed_cities=count(model.closed),
                                )
end

function                                city_observations(model)
                                rows                                =                                NamedTuple[]
                                for                                city                                in                                1:model.C
                                ids                                =                                collect(ids_in_position(city,                                model))
                                push!(rows,                                (
time=model.day, city, susceptible=count(id -> model[id].status == :S, ids),
infected=count(id -> model[id].status == :I, ids),
recovered=count(id -> model[id].status == :R, ids),
living=length(ids), deaths=model.deaths_by_city[city], closed=model.closed[city],
                                ))
                                end
                                return                                rows
end

function                                collect_history(model,                                steps;                                by_city=false)
                                rows                                =                                by_city ? city_observations(model) : [observe(model)]
                                for                                _                                in                                1:steps
                                advance!(model)
                                append!(rows, by_city ? city_observations(model) : [observe(model)])
                                end
                                return                                rows
end

function                                run_sir(;                                n_steps=100,                                by_city=false,                                kwargs...)
                                model                                =                                initialize_sir(;                                kwargs...)
                                return                                model,                                collect_history(model,                                n_steps;                                by_city)
end

end

```

Листинг 5 — полный модуль project/src/sir_model.jl.

```

using                                CairoMakie
using                                CSV
using                                DataFrames
using                                DrWatson
using                                Statistics

include(joinpath(@__DIR__,                                "sir_model.jl"))
using                                .SIRModel

set_theme!(Theme(font="DejaVu                                Sans",                                fontsize=16,                                linewidth=2))

scenario_data(name,                                file)                                =                                (mkpath(datadir(name));                                datadir(name,                                file))
scenario_plot(name,                                file)                                =                                (mkpath(plotsdir(name));                                plotsdir(name,                                file))
report_figure(file)                                =                                (mkpath(projectdir(".",                                "image",                                "results"));                                projectdir(".",                                "image",                                "results",                                file))

function                                save_figure(fig,                                scenario,                                filename;                                report_name=nothing)
                                save(scenario_plot(scenario,                                filename),                                fig)
                                isnothing(report_name) || save(report_figure(report_name),                                fig)
                                return                                fig
end

function                                sir_figure(frame;                                title="Динамика                                агентной                                модели                                SIR")
                                fig                                =                                Figure(size=(1040,                                620))

```

```

        ax = Axis(fig[1, 1]; title=title, xlabel="День", ylabel="Число агентов")
        lines!(ax, frame.time, frame.susceptible; label="S — восприимчивые", color=:royalblue)
        lines!(ax, frame.time, frame.infected; label="I — инфицированные", color=:firebrick)
        lines!(ax, frame.time, frame.recovered; label="R — выздоровевшие", color=:seagreen)
        lines!(ax, frame.time, frame.living; label="Живые", color=:gray35, linestyle=:dash)
        axislegend(ax; position=:rc)
        return fig
    end

    function save_frame(frame, scenario, filename)
        CSV.write(scenario_data(scenario, filename), frame)
        return frame
    end

    function epidemic_metrics(frame, initial_population)
        peak, idx = findmax(frame.infected_fraction)
        return (
            peak_fraction=peak, peak_day=frame.time[idx],
            final_infected=last(frame.infected_fraction),
            final_recovered=last(frame.recovered) / initial_population,
            deaths=last(frame.deaths), attack_rate=(initial_population - last(frame.susceptible)) / initial_population,
        )
    end

    function summarize_groups(frame, key)
        return combine(groupby(frame, key),
            :peak_fraction => mean => :mean_peak_fraction,
            :peak_day => mean => :mean_peak_day,
            :deaths => mean => :mean_deaths,
            :attack_rate => mean => :mean_attack_rate,
        )
    end
end

```

Листинг 6 — сбор статистики и общая визуализация.

4.3 Базовый опыт и число R_0

4.3.1 Базовый опыт агентной модели SIR

Модель запускается с параметрами методического пособия. Для сопоставления теории и эксперимента вычисляется $R_0 = \beta / \gamma$, где $\gamma = 1 / T_{inf}$, а затем измеряются день и величина фактического пика.

4.3.1.1 Подключение проекта

```

using DrWatson
@quickactivate "AgentSIRLab"
using DataFrames, JLD2
include(srcdir("experiment_tools.jl"))

```

4.3.1.2 Параметры и расчёт

```

params = (
    Ns=[1000, 1000, 1000], β_und=[0.5, 0.5, 0.5], β_det=[0.05, 0.05, 0.05],
    infection_period=14, detection_time=7, death_rate=0.02,
    reinfection_probability=0.1, Is=[0, 0, 1], seed=42,
)

```

```

)
model,          history_rows          =          run_sir(;          params...,          n_steps=100)
trajectory      =          save_frame(DataFrame(history_rows),          "sir-run-basic",          "trajectory.csv")
metrics         =          DataFrame([merge(epidemic_metrics(trajectory,          sum(params.Ns)),
(R0=basic_reproduction_number(params.β_und[1],          params.infection_period))),
save_frame(metrics,          "sir-run-basic",          "summary.csv")
jldsave(scenario_data("sir-run-basic",          "sir_basic_agent.jld2");          trajectory)
jldsave(scenario_data("sir-run-basic", "sir_basic_model.jld2"); metrics)

```

4.3.1.3 Визуализация

```

fig      =      sir_figure(trajectory;      title="Базовый      опыт:      R0      =      $(metrics.R0[1])"
save_figure(fig,      "sir-run-basic",      "sir-basic-dynamics.png";
report_name="sir-baseline-dynamics.png")
display(fig)
metrics

```

Листинг 7 — базовый литературный сценарий.

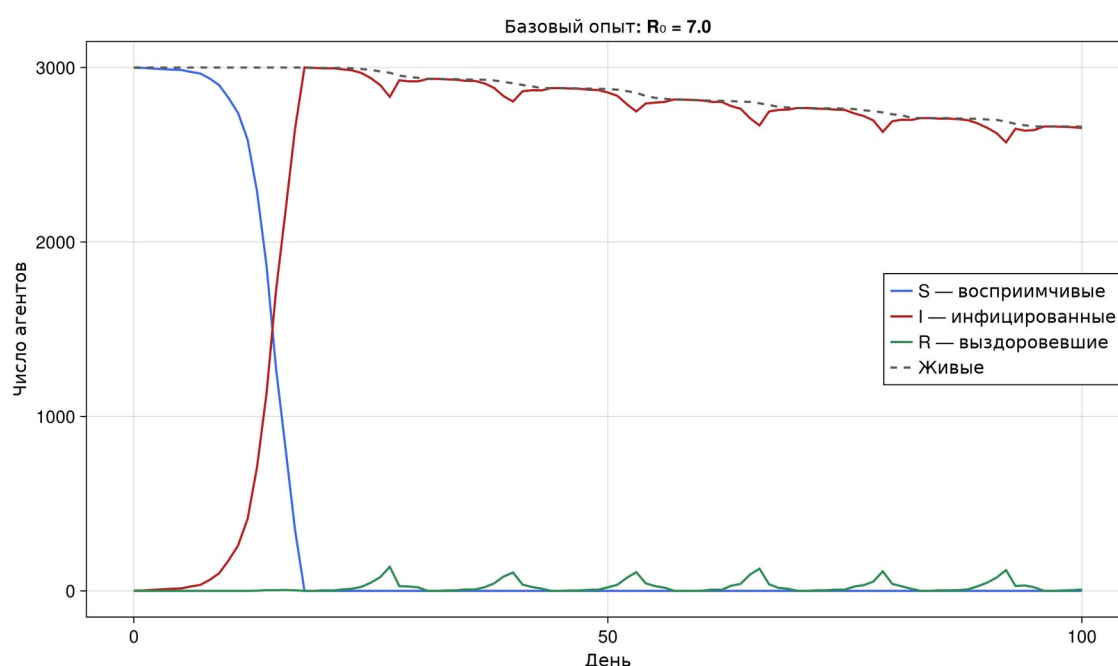


Рисунок 4: Базовая динамика SIR

Расчёт даёт $R_0 = 7$. Наблюдаемый пик достигается на 18-й день. Из-за высокого R_0 и вероятности повторного заражения система быстро охватывает все три города, а к 100-му дню зарегистрировано 339 смертей.

4.4 Параметрический вариант

4.4.1 Параметрический вариант базового опыта

Требование §4.2 проверяется на независимом наборе сочетаний заразности и продолжительности болезни. Для каждого сочетания сохраняются траектория и эпидемиологические показатели.

4.4.1.1 Подключение проекта

```
using                               DrWatson
@quickactivate                      "AgentSIRLab"
using                               DataFrames
include(srcdir("experiment_tools.jl"))
```

4.4.1.2 Серия прогонов

```
settings = [
    (name="мягкий",      beta=0.18,    period=10,    seed=71),
    (name="средний",     beta=0.32,    period=12,    seed=72),
    (name="интенсивный", beta=0.50,    period=14,    seed=73),
]
all_trajectories = DataFrame()
summary_rows = NamedTuple{(), Tuple{}}()
for p in settings
    rows = run_sir(Ns=[700, 700, 700],
        beta_det=fill(p.beta, 10, 3),
        infection_period=p.period,
        detection_time=min(7, p.period),
        death_rate=0.02,
        reinfection_probability=0.1,
        Is=[0, 0, 2],
        seed=p.seed,
        n_steps=120)
    frame = DataFrame{typeof(rows)}(rows)
    frame.scenario = p.name
    append!(all_trajectories, frame)
    push!(summary_rows, merge((scenario=p.name, beta=p.beta,
        infection_period=p.period, R0=basic_reproduction_number(p.beta, p.period)),
        epidemic_metrics(frame, 2100)))
end
save_frame(all_trajectories, "sir-run-basic__param", "trajectories.csv")
summary = save_frame(DataFrame(summary_rows), "sir-run-basic__param", "summary.csv")
```

4.4.1.3 Сравнение доли инфицированных

```
fig = Figure(size=(1040, 620))
ax = Axis(fig[1, 1], title="Параметрические варианты SIR", xlabel="День",
    ylabel="Доля инфицированных")
for (p, colour) in zip(settings, (:royalblue, :darkorange, :firebrick))
    subset = filter(:scenario ==> p.name, all_trajectories)
    lines!(ax, subset.time, subset.infected_fraction; label="$p.name, R0=$(round(p.beta*p.period; digits=2))", color=colour)
end
axislegend(ax, position=:rt)
save_figure(fig, "sir-run-basic__param", "parameter-comparison.png";
    report_name="sir-parameter-comparison.png")
display(fig)
summary
```

Листинг 8 — параметрический вариант §4.2.

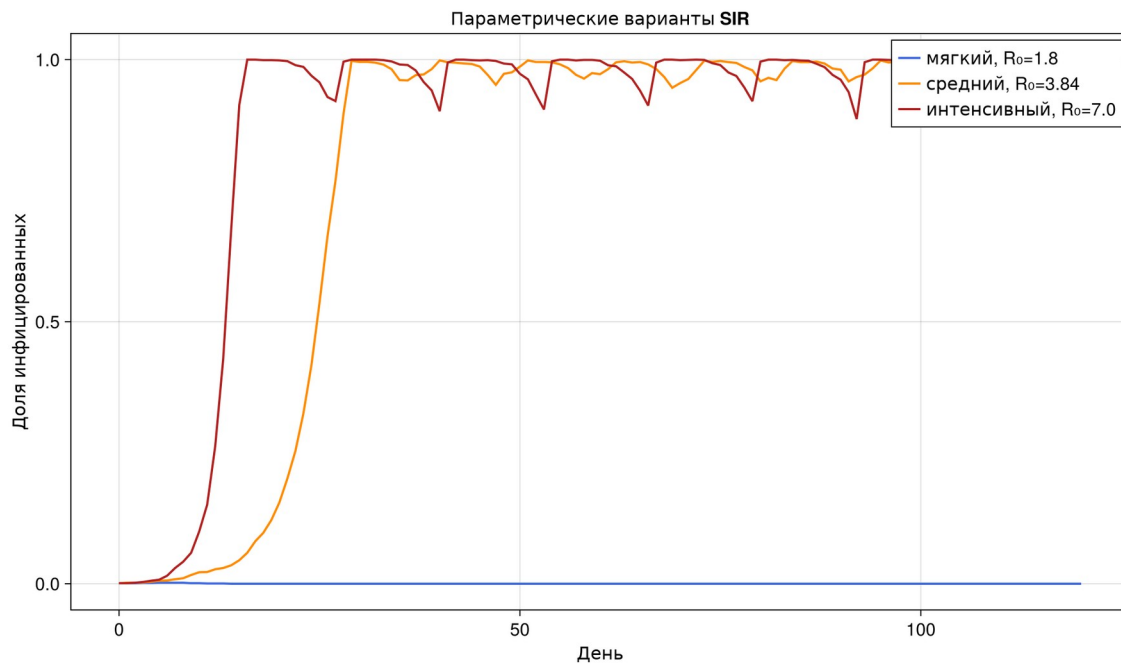


Рисунок 5: Три параметрических режима

При $R_0=1,8$ начальная цепочка прекращается и пик составляет 0,19%. Режимы с $R_0=3,84$ и $R_0=7$ формируют крупную эпидемию, что показывает влияние стохастического старта даже выше теоретического порога.

4.5 Наблюдаемый порог эпидемии

4.5.1 Сканирование коэффициента заразности

Для каждого $\beta \in [0.1, 1.0]$ выполняются три стохастических прогона. Эпидемией считается опыт, где пик превышает 5% начальной популяции.

4.5.1.1 Подключение проекта

```
using                                     DrWatson
@quickactivate                           "AgentSIRLab"
using                                     DataFrames,
include(srcdir("experiment_tools.jl")) Statistics
```

4.5.1.2 Полный факторный расчёт

```
rows = NamedTuple{(), Tuple{}}
for beta in 0.1:0.1:1.0, seed in 1000, 1000, 1000
    history = run_sir(Ns=[1000, 1000, 1000], beta_und=fill(beta, 3),
        beta_det=fill(beta / 10, 3), infection_period=14, detection_time=7,
        death_rate=0.02, reinfection_probability=0.1, Is=[0, 0, 1],
        seed=seed, n_steps=100)
    push!(rows, merge((beta=beta, seed=seed, R0=basic_reproduction_number(beta, 14)),
        epidemic_metrics(DataFrame(history), 3000)))
end
all_runs = save_frame(DataFrame(rows), "sir-scan-beta", "beta-scan-all.csv")
summary = combine(groupby(all_runs, [:beta, :R0]), :peak_fraction => mean => :mean_peak_fraction,
```



```

summary.epidemic          = summary.mean_peak_fraction      .>      0.05
save_frame(summary,       "sir-scan-beta",                  "beta-scan-summary.csv")
observed_threshold        = minimum(summary.beta[summary.epidemic])
threshold                 = DataFrame(observed_beta=[observed_threshold],
                                     observed_R0=[14 * observed_threshold], theoretical_beta=[1 / 14], theoretical_R0=[1.0])
save_frame(threshold, "sir-scan-beta", "epidemic-threshold.csv")

```

4.5.1.3 Порог и нагрузка

```

fig = Figure(size=(1040, 720))
ax1 = Axis(fig[1, 1], title="Порог эпидемии и смертность", xlabel="β", ylabel="Пиковая доля I")
scatterlines!(ax1, summary.beta, summary.mean_peak_fraction; marker=:circle, color=:firebrick)
hlines!(ax1, [0.05]; linestyle=:dash, color=:gray40, label="Критерий 5%")
vlines!(ax1, [observed_threshold]; linestyle=:dot, color=:black, label="Наблюдаемый порог")
axislegend(ax1; position=:lt)
ax2 = Axis(fig[2, 1], xlabel="β", ylabel="Среднее число умерших")
scatterlines!(ax2, summary.beta, summary.mean_deaths; marker=:diamond, color=:purple)
save_figure(fig, "sir-scan-beta", "beta-scan.png"; report_name="sir-beta-threshold.png")
display(fig)
threshold

```

Листинг 9 — тридцать прогонов сканирования β .

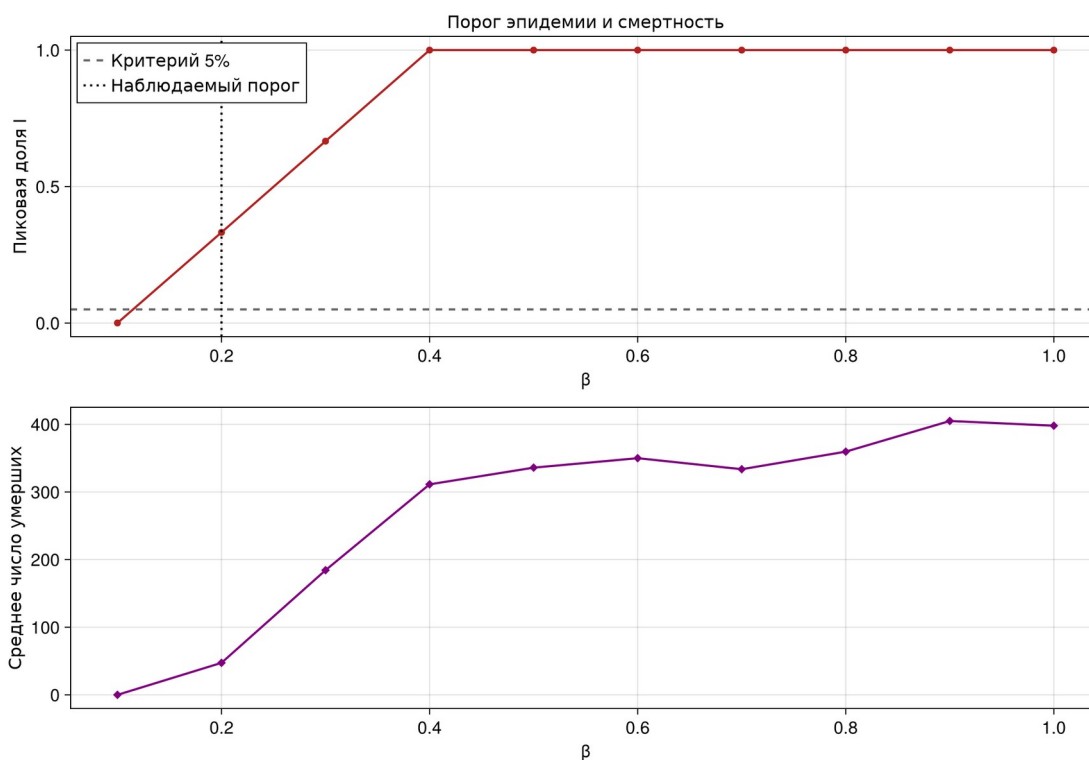


Рисунок 6: Порог эпидемии и смертность

В исследованной сетке минимальным значением с усреднённым пиком выше 5% оказалось $\beta=0,2$, то есть $R_0=2,8$. Теоретическая граница $R_0=1$ соответствует $\beta \approx 0,0714$, но единичный источник инфекции может стохастически исчезнуть до образования устойчивой цепочки.

4.6 Гетерогенность городов

4.6.1 Неоднородность городов

Сравниваются однородная система и три города с различными коэффициентами передачи. Статистика собирается отдельно для каждого города.

4.6.1.1 Подключение проекта

```
using DrWatson
@quickactivate "AgentSIRLab"
using DataFrames
include(srcdir("experiment_tools.jl"))
```

4.6.1.2 Городские траектории

```
cases = [
    (case="однородная", beta=[0.5, 0.5, 0.5], seed=91),
    (case="неоднородная", beta=[0.22, 0.50, 0.82], seed=91),
]

city_data = DataFrame{
    case, rows, in cases,
    β_det=case.beta ./ 10, infection_period=14, detection_time=7,
    death_rate=0.02, reinfection_probability=0.1, Is=[0, 0, 3],
    migration_rates=migration_matrix(3, 0.04), seed=case.seed,
    n_steps=120, by_city=true)

frame = DataFrame{
    case, case.case, case.case,
    frame.infected_fraction = frame.infected ./ max.(frame.living, 1)
}

append!(city_data, frame)

end

save_frame(city_data, "sir-city-heterogeneity", "city-trajectories.csv")

summary = combine(groupby(city_data, :case, :city),
    [:infected, :time] => ((infected, time) -> time[argmax(infected)]) => :peak_fraction,
    :deaths => maximum => :deaths)

save_frame(summary, "sir-city-heterogeneity", "city-summary.csv")
```

4.6.1.3 Отдельные кривые для городов

```
fig = Figure(size=(1100, 760))

for (column, case) in enumerate(cases)
    ax = Axis(fig[1, column], title=case.case, xlabel="День", ylabel="Доля I")
    for (city, colour) in zip(1:3, (:royalblue, :darkorange, :firebrick))
        subset = filter([:case, :city] => (c, n) -> c == case.case && n == city, city_data)
        lines!(ax, subset.time, subset.infected_fraction; label="Город $city", color=colour)
    end
    axislegend(ax; position=:rt)
end

save_fig(fig, "sir-city-heterogeneity", "city-heterogeneity.png";
    report_name="sir-city-heterogeneity.png")

display(fig)

comparison = Figure(size=(980, 570))

axis = Axis(comparison[1, 1], title="Дни городских пиков",
    xlabel="Город", ylabel="День пика")
```

```

for (offset, case, colour) in ((-0.16, "однородная", :royalblue),
                              (0.16, "неоднородная", :darkorange))
    subset = filter(:case == case, summary)
    barplot!(axis, subset.city, subset.peak_day,
              width=0.29, color=colour, label=case)
end
axis.xticks = (1:3, ["1", "2", "3"])
axislegend(axis, position=:lt)
save_figure(comparison, "sir-city-heterogeneity", "city-peak-days.png";
            report_name="sir-city-peak-days.png")
summary

```

Листинг 10 — однородный и неоднородный городские опыты.

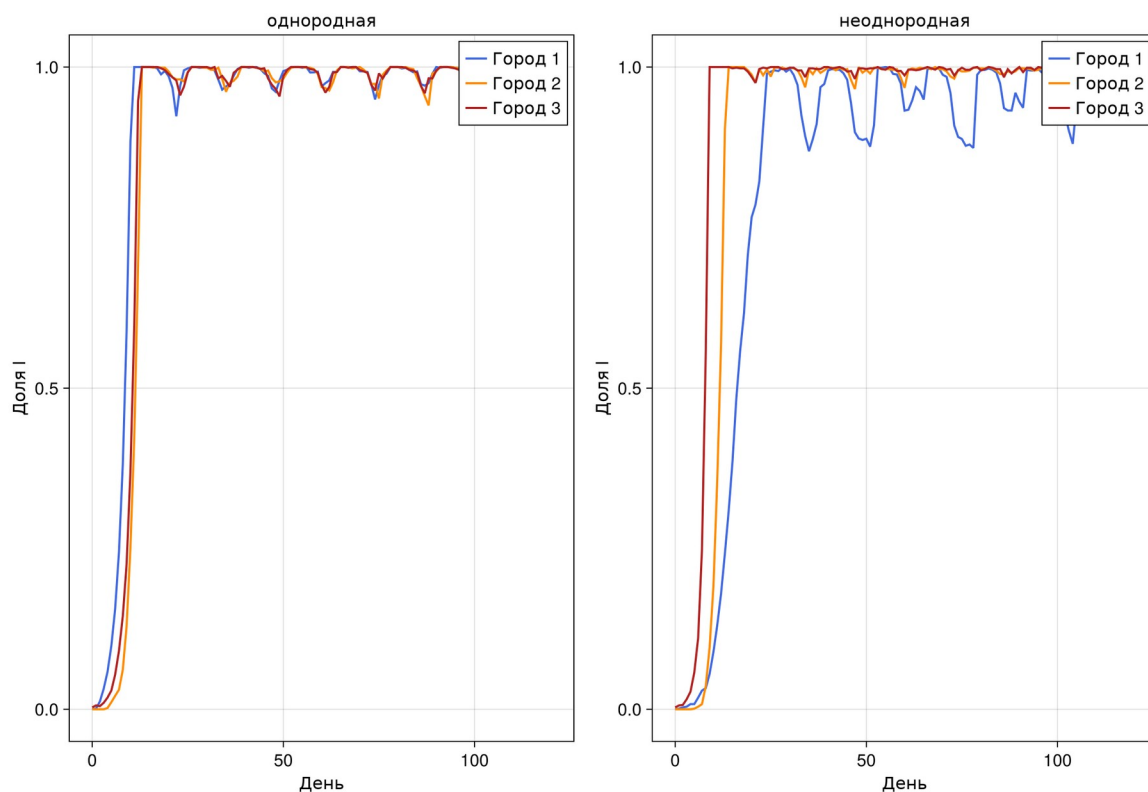


Рисунок 7: Динамика по каждому городу

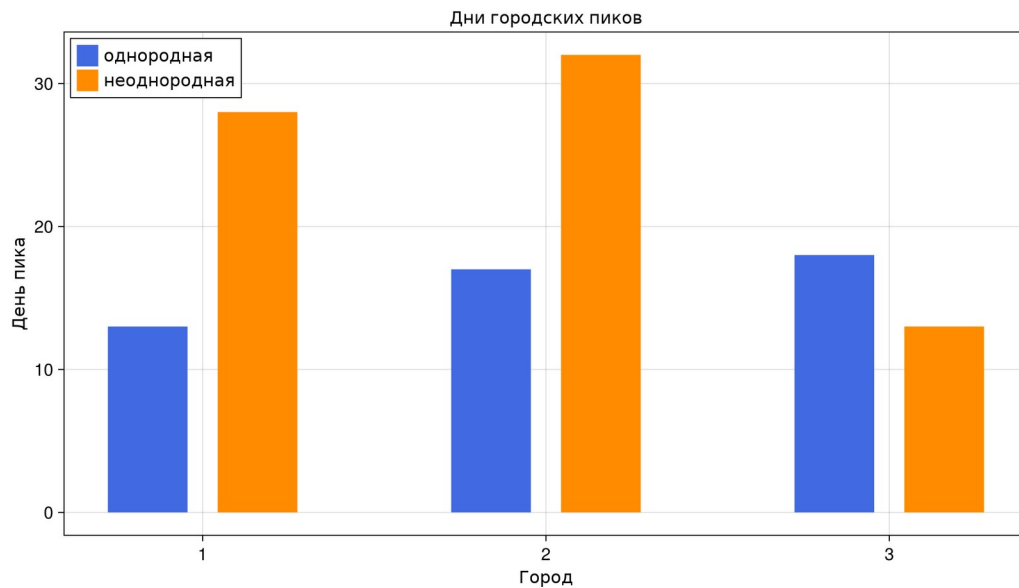


Рисунок 8: Сравнение дней городских пиков

При коэффициентах 0,22; 0,50 и 0,82 город с наибольшей заразностью достигает пика первым, а город с $\beta=0,22$ — последним. Различие сохраняется при общей матрице миграции и одинаковой продолжительности болезни.

4.7 Интенсивность миграции

4.7.1 Влияние миграции

Инфекция начинается в первом городе. Для каждой интенсивности измеряются глобальный пик и первый день появления инфекции во втором и третьем городах.

4.7.1.1 Подключение проекта

```
using                                     DrWatson
@quickactivate                           "AgentSIRLab"
using                                   DataFrames,
include(scrdir("experiment_tools.jl")) Statistics
```

4.7.1.2 Сканирование миграции

```
rows                                     = NamedTuple[]
for intensity in 0.0:0.1:0.5, seed in 1000, 1000, 1000, in 42:44
    model = initialize_sir(Ns=[1000, 1000, 1000],
        beta_det=fill(0.05, 3), infection_period=14, detection_time=7,
        death_rate=0.02, reinfection_probability=0.1, Is=[3, 0, 0],
        migration_rates=migration_matrix(3, intensity), seed=seed)
    global_rows = [observe(model)]
    city_rows = city_observations(model)
    for _ in 1:150
        advance!(model)
        push!(global_rows, observe(model))
        append!(city_rows, city_observations(model))
    end
end
```

```

global_frame, city_frame = DataFrame(global_rows), DataFrame(city_rows)
metrics = epidemic_metrics(global_frame, 3000)
arrival(city) = begin
subset = filter(:city == (city), city_frame)
index = findfirst(>(0), subset.infected)
isnothing(index) ? missing : subset.time[index]
end
push!(rows, merge((migration_intensity=intensity, seed=seed,
arrival_city_2=arrival(2), arrival_city_3=arrival(3)), metrics))
end
all_runs = save_frame(DataFrame(rows), "sir-migration-effect", "migration-scan-all.csv")
summary = combine(groupby(all_runs, :migration_intensity),
:peak_day => mean => :mean_peak_day,
:peak_fraction => mean => :mean_peak_fraction,
:arrival_city_2 => (x -> mean(skipmissing(x))) => :mean_arrival_city_2,
:arrival_city_3 => (x -> mean(skipmissing(x))) => :mean_arrival_city_3)
save_frame(summary, "sir-migration-effect", "migration-summary.csv")
best = summary[argmin(summary.mean_peak_day), :]

```

4.7.1.3 Скорость распространения и пик

```

fig = Figure(size=(1050, 760))
ax1 = Axis(fig[1, 1], title="Миграция и время развития эпидемии", ylabel="День")
scatterlines!(ax1, summary.migration_intensity, summary.mean_peak_day; marker=:circle, label="День пика", color=:firebrick)
scatterlines!(ax1, summary.migration_intensity, summary.mean_arrival_city_3; marker=:rect, label="Приход в город 3",
color=:royalblue)
axislegend(ax1; position=:rt)
ax2 = Axis(fig[2, 1], xlabel="Интенсивность миграции", ylabel="Пиковая доля I")
scatterlines!(ax2, summary.migration_intensity, summary.mean_peak_fraction; marker=:diamond, color=:purple)
save_figure(fig, "sir-migration-effect", "migration-effect.png"; report_name="sir-migration-effect.png")
display(fig)
DataFrame([NamedTuple(best)])

```

Листинг 11 — исследование шести интенсивностей миграции.

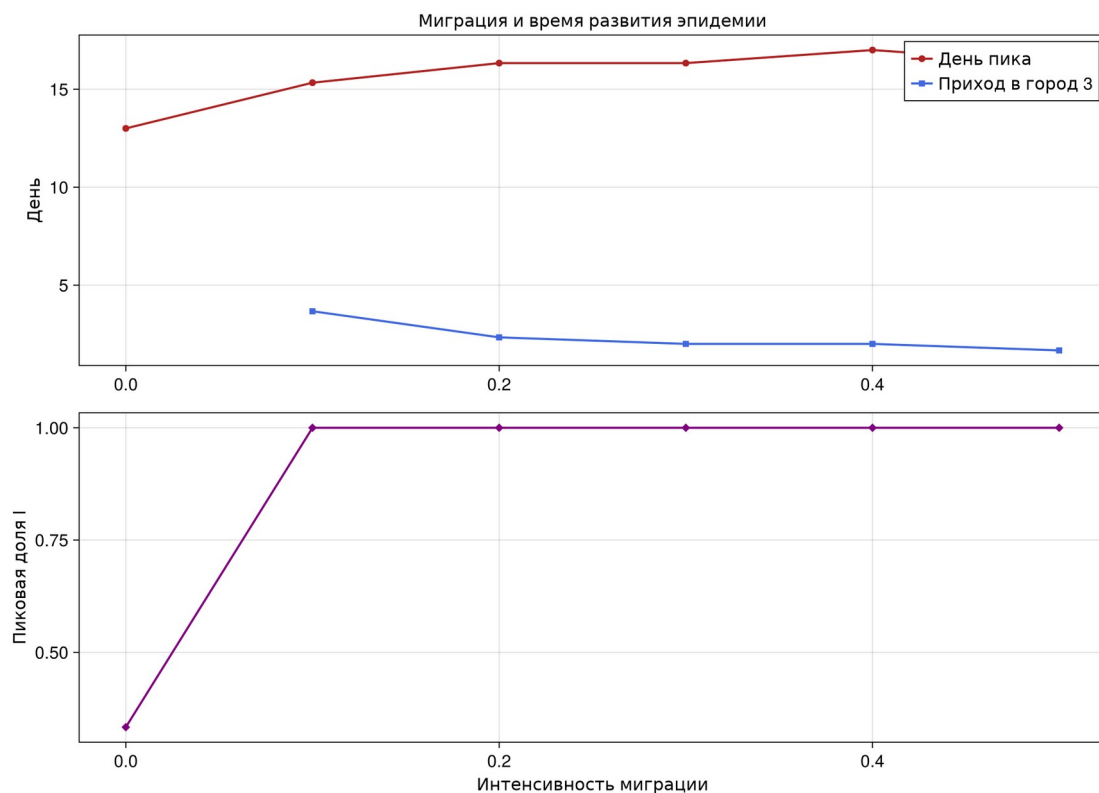


Рисунок 9: Миграция, день распространения и пик

Без миграции эпидемия остаётся в первом городе, поэтому глобальная доля имеет верхнюю границу около одной трети. При интенсивности 0,1 инфекция приходит в остальные города в среднем к четвёртому дню. Минимальный средний день прихода в третий город получен при интенсивности 0,5.

4.8 Карантинные меры

4.8.1 Карантинные меры

Город закрывается для исходящей миграции, когда доля инфицированных достигает 5%. Эффект оценивается по пику, смертности и времени проникновения инфекции.

4.8.1.1 Подключение проекта

```
using                               DrWatson
@quickactivate                       "AgentSIRLab"
using                               DataFrames,
include(srcdir("experiment_tools.jl")) Statistics
```

4.8.1.2 Парное сравнение

```
rows = NamedTuple{()
example_frames = Dict{Bool,DataFrame}()
for quarantine in (false, true), seed in 101:105
    history = run_sir(Ns=[1000, 1000, 1000], β_und=fill(0.5, 3),
    β_det=fill(0.05, 3), infection_period=14, detection_time=7,
    death_rate=0.02, reinfection_probability=0.1, Is=[5, 0, 0],
    migration_rates=migration_matrix(3, 0.12), seed=seed,
    quarantine_enabled=quarantine, quarantine_threshold=0.05, n_steps=140)
    frame = DataFrame(history)
    push!(rows, merge((quarantine=quarantine, seed=seed), epidemic_metrics(frame, 3000),
    (quarantine_days=count(>0), frame.closed_cities),))
end
all_runs = save_frame(DataFrame(rows), "sir-quarantine", "quarantine-all.csv")
summary = combine(groupby(all_runs, :quarantine),
    :peak_fraction => mean => :mean_peak_fraction,
    :peak_day => mean => :mean_peak_day,
    :deaths => mean => :mean_deaths,
    :attack_rate => mean => :mean_attack_rate,
    :quarantine_days => mean => :mean_quarantine_days)
save_frame(summary, "sir-quarantine", "quarantine-summary.csv")
```

4.8.1.3 Динамика контрольного прогона

```
fig = Figure(size=(1040, 620))
ax = Axis(fig[1, 1], title="Эффект закрытия городов", xlabel="День", ylabel="Доля инфицированных")
for (flag, colour, label) in ((false, :firebrick, "Без карантина"), (true, :royalblue, "С карантином"))
    frame = example_frames[flag]
    lines!(ax, frame.time, frame.infected_fraction; label, color=colour)
end
axislegend(ax, position=:rt)
```

```

save_figure(fig, "sir-quarantine", "quarantine-effect.png";
report_name="sir-quarantine-effect.png")

display(fig)

metrics_fig = Figure(size=(980, 570))
peak_axis = Axis(metrics_fig[1, 1], title="Средний пик", ylabel="Доля I")
death_axis = Axis(metrics_fig[1, 2], title="Средняя смертность", ylabel="Агенты")
labels = ["нет", "есть"]
barplot!(peak_axis, [1, 2], summary.mean_peak_fraction; color=[:firebrick, :royalblue])
barplot!(death_axis, [1, 2], summary.mean_deaths; color=[:firebrick, :royalblue])
peak_axis.xticks = ([1, 2], labels)
death_axis.xticks = ([1, 2], labels)
save_figure(metrics_fig, "sir-quarantine", "quarantine-metrics.png";
report_name="sir-quarantine-metrics.png")

summary

```

Листинг 12 — парное сравнение пяти seed с карантином и без него.

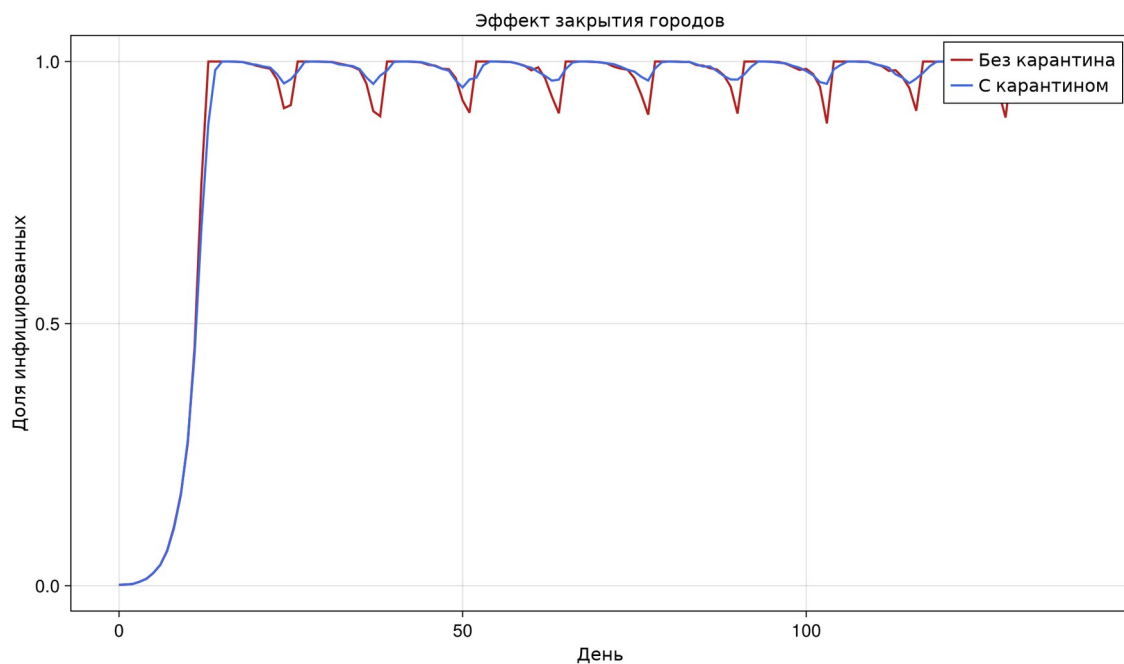


Рисунок 10: Сравнение режимов закрытия городов

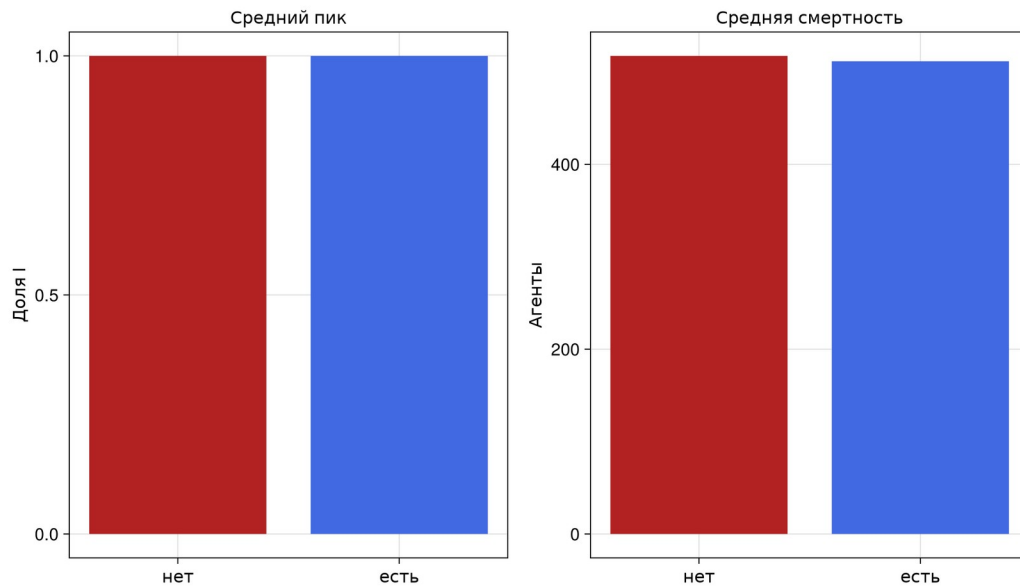


Рисунок 11: Пик и смертность в двух режимах

При исходных $R_0=7$ закрытие только исходящей миграции после достижения 5% не останавливает внутригородскую передачу: средний пик остаётся близок к 100%. Средняя смертность уменьшается с 517,4 до 511,6. Следовательно, такая мера должна сочетаться с ранним выявлением и уменьшением контактов.

4.9 Многокритериальная оптимизация

```

sir_optimize_parameters.jl
1  # Многокритериальная оптимизация
2  #
3  # Эволюционный алгоритм одновременно минимизирует смертность и пик инфекции.
4  # В допустимое множество входят решения с пиком не выше 30%; недопустимые
5  # решения получают штраф. Orbot использует Pareto-ранг и crowding distance.
6
7  ## Подключение проекта
8  using DrWatson
9  @quickactivate "AgentSIRLab"
10 using DataFrames, Random, Statistics
11 include(sourcedir("experiment_tools.jl"))
12
13 ## Кодирование решения и целевая функция
14 const NOPT = 1500
15 decode(x) = (detection_time=clamp(round(Int, x[1]), 2, 10),
16             beta_detected=clamp(x[2], 0.01, 0.18),
17             migration=clamp(x[3], 0.0, 0.25),
18             quarantine_threshold=clamp(x[4], 0.03, 0.18))
19
20 function evaluate(x)
21     p = decode(x)
22     metrics = NamedTuple{()}{Float64, Int64}
23     for seed in (251, 252)
24         rows = run_sir(Ns=[500, 500, 500], p_und=fill(0.5, 3),
25                     p_det=fill(p.beta_detected, 3), infection_period=14,
26                     detection_time=p.detection_time, death_rate=0.02,
27                     reinfection_probability=0.1, Is=[4, 0, 0],
28                     migration_rates=migration_matrix(3, p.migration), seed=seed,
29                     quarantine_enabled=true, quarantine_threshold=p.quarantine_threshold,
30                     n_steps=120)
31         push!(metrics, epidemic_metrics(DataFrame(rows), NOPT))
32     end
33     peak = mean(m.peak_fraction for m in metrics)
34     deaths = mean(m.deaths for m in metrics) / NOPT
35     violation = max(0.0, peak - 0.30)
36     return (death_fraction=deaths + 10violation, peak_fraction=peak + 10violation,
37           raw_death_fraction=deaths, raw_peak_fraction=peak)
38 end
39
40 dominates(a, b) = all(a .<= b) && any(a .< b)
41 function pareto_ranks(values)
42     remaining_ranks = collect{eachindex(values)} .zeros{Int, length(values)}

```

Рисунок 12: Эволюционный сценарий в VS Code

4.9.1 Многокритериальная оптимизация

Эволюционный алгоритм одновременно минимизирует смертность и пик инфекции. В допустимое множество входят решения с пиком не выше 30%; недопустимые решения получают штраф. Отбор использует Парето-ранг и crowding distance.

4.9.1.1 Подключение проекта

```
using                                     DrWatson
@quickactivate                           "AgentSIRLab"
using DataFrames,                       Random,
include(srcdir("experiment_tools.jl")) Statistics
```

4.9.1.2 Кодирование решения и целевая функция

```
const NOPT = 1500
decode(x) = (detection_time=clamp(round{Int, x[1]}, 2, 10),
             beta_detected=clamp(x[2], 0.01, 0.18),
             migration=clamp(x[3], 0.0, 0.25),
             quarantine_threshold=clamp(x[4], 0.03, 0.18))

function evaluate(x)
    p = decode(x)
    metrics = NamedTuple{(), Tuple{}}()
    for seed in (251, 252)
        rows = run_sir(Ns=[500, 500, 500], β_und=fill(0.5, 3),
                      β_det=fill(p.beta_detected, 3), infection_period=14,
                      detection_time=p.detection_time, death_rate=0.02,
                      reinfection_probability=0.1, Is=[4, 0, 0],
                      migration_rates=migration_matrix(3, p.migration), seed=seed,
                      quarantine_enabled=true, quarantine_threshold=p.quarantine_threshold,
                      n_steps=120)
        push!(metrics, epidemic_metrics(DataFrame(rows), NOPT))
    end
    peak = mean(m.peak_fraction for m in metrics)
    deaths = mean(m.deaths for m in metrics) / NOPT
    violation = max(0.0, peak - 0.30)
    return (death_fraction=deaths + 10violation, peak_fraction=peak + 10violation,
            raw_death_fraction=deaths, raw_peak_fraction=peak)
end

dominates(a, b) = all(a .<= b) && any(a .< b)
function pareto_ranks(values)
    remaining, ranks, rank = collect(eachindex(values)), zeros{Int, length(values)}, 1
    while !isempty(remaining)
        front = [i for i in remaining if !any(j -> j != i && dominates(values[j], values[i]), remaining)]
        ranks[front] = rank
        remaining = setdiff(remaining, front)
        rank += 1
    end
    return ranks
end

function crowding(values, ranks)
    distance = zeros{length(values)}
```

```

        for rank in unique(ranks)
            front = findall(==(rank), ranks)
            length(front) <= 2 && (distance[front] .== Inf; continue)
            for objective in 1:2
                ordered = sort(front; by=i -> values[i][objective])
                distance[ordered[[1, end]]] .== Inf
                span = values[ordered[end]][objective] - values[ordered[1]][objective]
                span == 0 && continue
                for k in 2:length(ordered)-1
                    distance[ordered[k]] += (values[ordered[k+1]][objective] - values[ordered[k-1]][objective]) / span
                end
            end
        end
        return distance
end

```

4.9.1.3 Эволюционный поиск NSGA-II

```

rng = Xoshiro(404)
random_candidate() = [rand(rng, 2:10), rand(rng) * 0.17 + 0.01,
                      rand(rng) * 0.25, rand(rng) * 0.15 + 0.03]
population = [random_candidate() for _ in 1:18]
history_rows = NamedTuple{(), Tuple{}}()
for generation in 1:7
    global population
    scores = [evaluate(x) for x in population]
    objectives = [(s.death_fraction, s.peak_fraction) for s in scores]
    ranks = pareto_ranks(objectives)
    distance = crowding(objectives, ranks)
    for (index, (x, score)) in enumerate(zip(population, scores))
        p = decode(x)
        push!(history_rows, merge((generation=generation, candidate=index, rank=ranks[index],
                                   crowding=distance[index]), p, score))
    end
    order = sortperm(eachindex(population); by=i -> (ranks[i], -distance[i]))
    parents = population[order[1:9]]
    children = Vector{Float64}{}
    while length(children) < 9
        a, b = rand(rng, parents, 2)
        λ = rand(rng)
        child = λ * a + (1 - λ) * b
        child .+= [randn(rng) * 0.7, randn(rng) * 0.015, randn(rng) * 0.025, randn(rng) * 0.015]
        push!(children, collect(Float64, child))
    end
    population = vcat(copy.(parents), children)
end
history = save_frame(DataFrame(history_rows), "sir-optimize-parameters", "optimization-history.csv")
final = filter(:generation => == (maximum(history.generation)), history)
pareto = filter(:rank => == (1), final)
save_frame(pareto, "sir-optimize-parameters", "pareto-front.csv")
feasible = filter(:raw_peak_fraction => <= (0.30), pareto)
selection_pool = isempty(feasible) ? pareto : feasible
best = selection_pool[argmin(selection_pool.raw_death_fraction), :]
save_frame(DataFrame([NamedTuple(best)]), "sir-optimize-parameters", "optimization-best.csv")

```

4.9.1.4 Парето-фронт

```

fig = Figure(size=(1000, 660))
ax = Axis(fig[1, 1], title="Многокритериальная оптимизация SIR",
           xlabel="Пиковая доля инфицированных", ylabel="Доля умерших")
scatter!(ax, final.raw_peak_fraction, final.raw_death_fraction;
         color=final.rank, colormap=:viridis, markersize=14)

```

```

scatter!(ax,
          pareto.raw_peak_fraction,
          pareto.raw_death_fraction;
          color=:firebrick, markersize=20, marker=:star5, label="Парето-фронт")
vlines!(ax, [0.30];
         linestyle=:dash, color=:gray30, label="Ограничение 30%",
         position=:rt)
axislegend(ax,
           "sir-optimize-parameters",
           "pareto-front.png";
           report_name="sir-pareto-optimization.png")
save_figure(fig,
             "pareto-front.png";
             report_name="sir-pareto-optimization.png")
display(fig)
DataFrame([NamedTuple(best)])

```

Листинг 13 — эволюционный алгоритм с Парето-рангом.

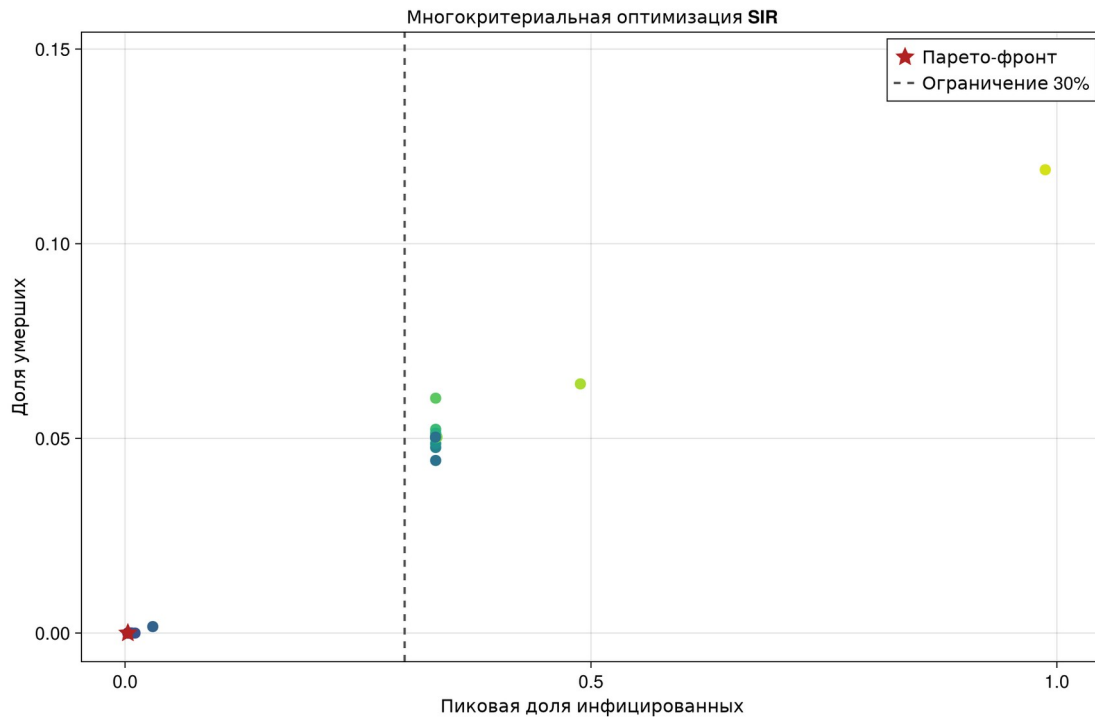


Рисунок 13: Решения и Парето-фронт

Поиск изменяет день выявления, заразность выявленных случаев, миграцию и порог карантина. Лучшее допустимое решение: выявление на 2-й день, $\beta_{det}=0,0160$, интенсивность миграции 0,2032 и порог закрытия 0,1020. В двух проверочных прогонах средний пик составил 0,3%, смертей не было.

4.10 Сводный анализ CSV

4.10.1 Сводный анализ сохранённых результатов

Этот сценарий не повторяет симуляции. Он читает CSV-файлы базового опыта, сканирования β , миграции и карантина и строит итоговую панель.

4.10.1.1 Загрузка рассчитанных таблиц

```

using
@quickactivate
using CSV, CairoMakie,
include(scrdir("experiment_tools.jl"))
DrWatson
"AgentSIRLab"
DataFrames

```

```

basic = CSV.read(datadir("sir-run-basic", "trajectory.csv"), DataFrame)
beta = CSV.read(datadir("sir-scan-beta", "beta-scan-summary.csv"), DataFrame)
migration = CSV.read(datadir("sir-migration-effect", "migration-summary.csv"), DataFrame)
quarantine = CSV.read(datadir("sir-quarantine", "quarantine-summary.csv"), DataFrame)

```

4.10.1.2 Комплексная визуализация

```

fig = Figure(size=(1200, 900))
ax1 = Axis(fig[1, 1], title="Базовая траектория", xlabel="День", ylabel="Агенты")
lines!(ax1, basic.time, basic.infected; color=:firebrick, label="I")
lines!(ax1, basic.time, basic.recovered; color=:seagreen, label="R")
axislegend(ax1)
ax2 = Axis(fig[1, 2], title="Попор зараженности", xlabel="β", ylabel="Пиковая доля I")
scatterlines!(ax2, beta.beta, beta.mean_peak_fraction; marker=:circle, color=:purple)
hlines!(ax2, [0.05]; linestyle=:dash, color=:gray40)
ax3 = Axis(fig[2, 1], title="Миграция", xlabel="Интенсивность", ylabel="День пика")
scatterlines!(ax3, migration.migration_intensity, migration.mean_peak_day; marker=:diamond, color=:royalblue)
ax4 = Axis(fig[2, 2], title="Карантин", xlabel="Режим", ylabel="Средний пик I")
barplot!(ax4, [1, 2], quarantine.mean_peak_fraction; color=[:firebrick, :royalblue])
ax4.xticks = ([1, 2], ["нет", "есть"])
save_figure(fig, "sir-visualize-dynamics", "comprehensive-analysis.png";
report_name="sir-comprehensive-analysis.png")

```

4.10.1.3 Таблица ключевых выводов

```

summary = DataFrame(
    indicator=["Базовый пик", "Минимальный исследованный β с эпидемией",
               "Минимальный средний день пика при миграции", "Пик при карантине"],
    value=[maximum(basic.infected_fraction),
           minimum(beta.beta[beta.mean_peak_fraction .> 0.05]),
           minimum(migration.mean_peak_day,
                  quarantine.mean_peak_fraction[quarantine.quarantine .== true][1]),
           ]
)
save_frame(summary, "sir-visualize-dynamics", "comprehensive-summary.csv")
display(fig)
summary

```

Листинг 14 — анализ ранее сохранённых CSV без повторной симуляции.

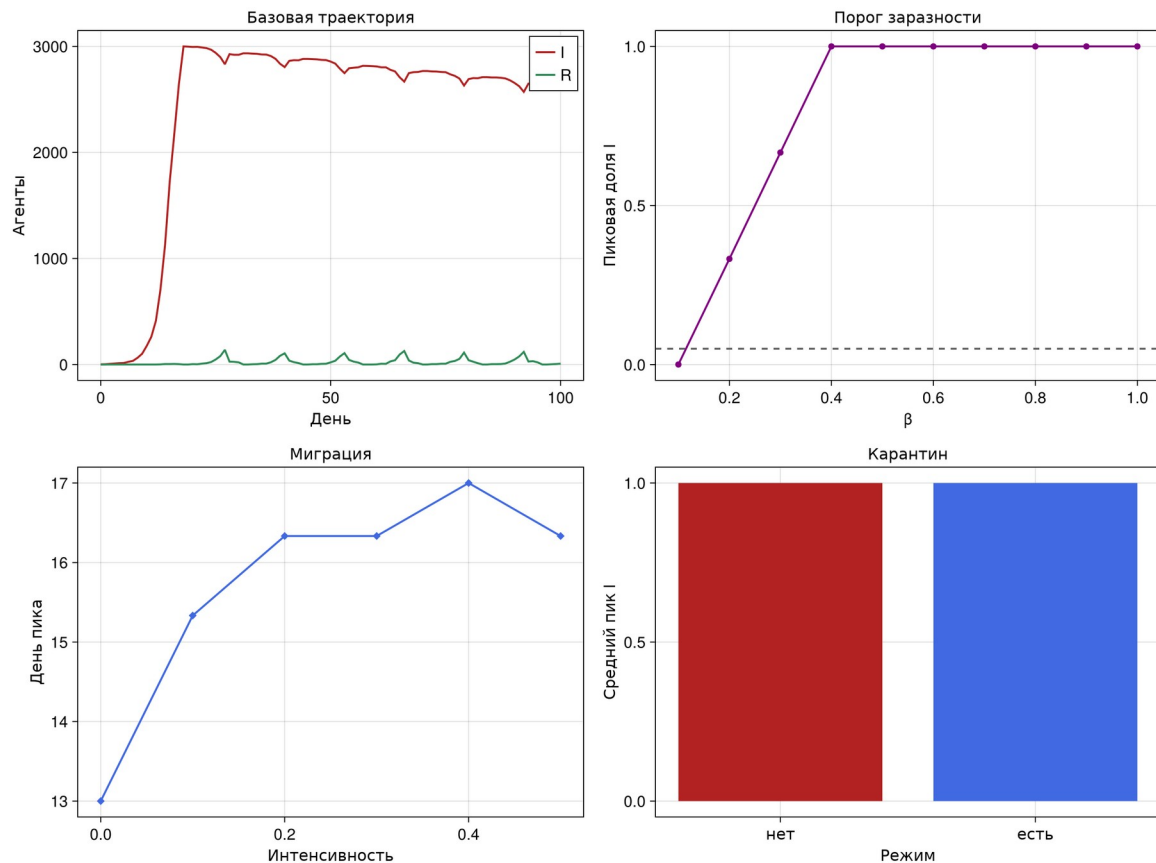


Рисунок 14: Комплексная панель результатов

Сводная панель соединяет четыре независимых результата и подтверждает, что управляемые параметры выявления сильнее влияют на пик, чем изолированное закрытие миграции при уже высокой распространённости.

4.11 Выполненные notebooks

Все восемь notebook выполнены ядром Julia. Каждая кодовая ячейка имеет непустой `execution_count`; ошибок выполнения нет, а графики сохранены внутри файлов.

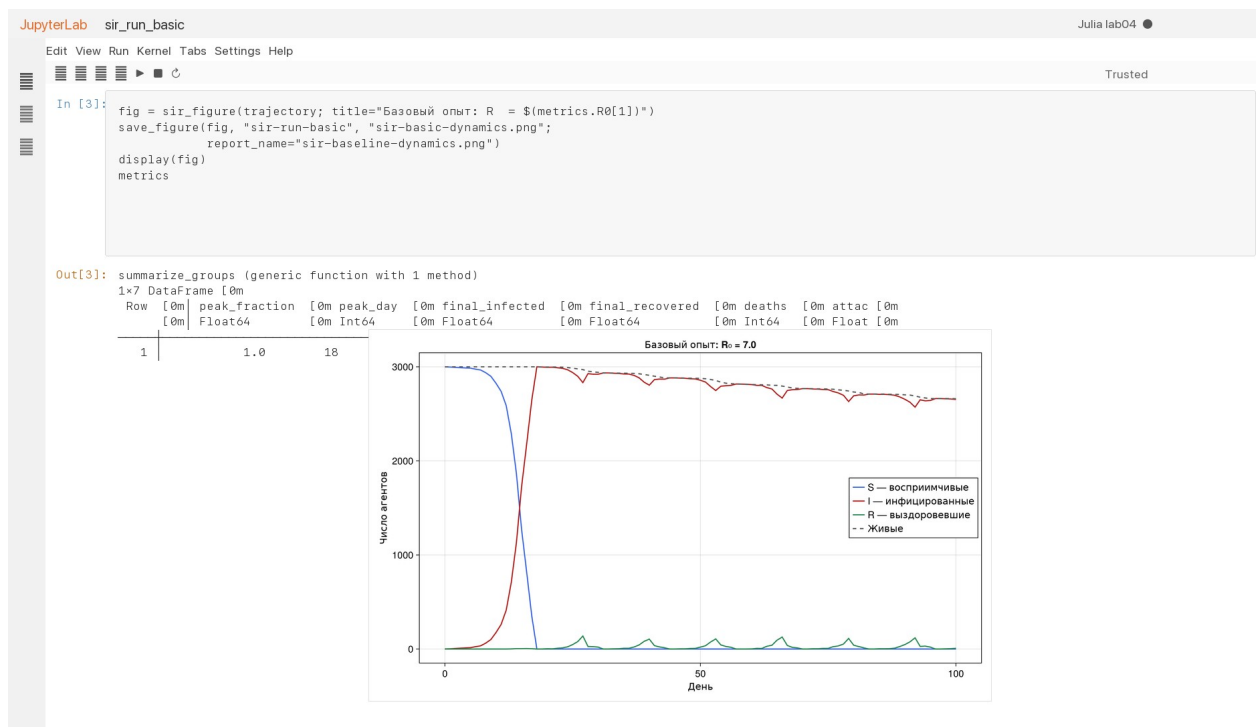


Рисунок 15: Базовый notebook



Рисунок 16: Notebook оптимизации

4.12 Автоматические проверки

```
using
using
using
@quickactivate
include(scrdir("sir_model.jl"))
using
```

Test
DataFrames
DrWatson
"AgentSIRLab"
.SIRModel

```

@testset "Migration" matrix" begin
    matrix = migration_matrix(3, 0.12)
    @test size(matrix) == (3, 3)
    @test all(isapprox.(vec(sum(matrix; dims=2)), fill(0.88, 3)))
    @test [matrix[i, i] for i in 1:3] == fill(0.88, 3)
    @test_throws ArgumentError migration_matrix(1, 0.1)
    @test_throws ArgumentError migration_matrix(3, 1.1)
end

@testset "Initialization and invariants" begin
    model = initialize_sir(Ns=[30, 40, 50], Is=[1, 2, 3],
        β_und=fill(0.3, 3), β_det=fill(0.03, 3), seed=7)
    @test total_count(model) == 120
    @test infected_count(model) == 6
    @test susceptible_count(model) == 114
    history = DataFrame(collect_history(model, 30))
    @test nrow(history) == 31
    @test history.time == 0:30
    @test all(history.susceptible .+ history.infected .+ history.recovered .== history.living)
    @test all(history.living .+ history.deaths .== 120)
    @test all((0 .<= history.infected_fraction) .& (history.infected_fraction .<= 1))
end

@testset "Reproducibility" begin
    kwargs = (Ns=[60, 60, 60], Is=[2, 0, 0], β_und=fill(0.45, 3),
        β_det=fill(0.045, 3), seed=44, n_steps=40)
    _, first = run_sir(; kwargs...)
    _, second = run_sir(; kwargs...)
    @test first == second
end

@testset "City statistics and quarantine" begin
    model = initialize_sir(Ns=[50, 50, 50], Is=[8, 0, 0],
        β_und=fill(0.5, 3), β_det=fill(0.05, 3),
        quarantine_enabled=true, quarantine_threshold=0.05,
        migration_rates=migration_matrix(3, 0.1), seed=19)
    @test model.closed[1]
    rows = DataFrame(collect_history(model, 10; by_city=true))
    @test nrow(rows) == 33
    @test sort(unique(rows.city)) == [1, 2, 3]
    @test all(rows.susceptible .+ rows.infected .+ rows.recovered .== rows.living)
end

@testset "Epidemiological quantities" begin
    @test basic_reproduction_number(0.5, 14) == 7.0
    @test basic_reproduction_number(1 / 14, 14) ≈ 1.0
    @test_throws ArgumentError initialize_sir(Ns=[10, 10], Is=[0, 0],
        β_und=[0.2], β_det=[0.02])
end

```

Листинг 15 — полный набор автоматических тестов.

```
rhktrlwmd@Mac % make test
julia --project=. test/runtests.jl
Test Summary: | Pass Total Time
Migration matrix | 5 5 0.3s
Initialization and invariants | 8 8 1.1s
Reproducibility | 1 1 0.0s
City statistics and quarantine | 4 4 0.3s
Epidemiological quantities | 3 3 0.0s
rhktrlwmd@Mac %
```

Рисунок 17: Успешное выполнение тестов

Проверена 21 характеристика: матрица миграции, входные ограничения, инициализация, баланс живых и умерших, воспроизводимость, городская статистика, карантин и формула R_0 .

5. Выводы

В работе реализована воспроизводимая агентная SIR-модель для трёх городов. Выполнены все пять основных сценариев, все шесть дополнительных заданий и параметрический вариант §4.2. На исследованной сетке наблюдаемый порог равен $\beta=0,2$, гетерогенность меняет порядок городских пиков, а миграция ускоряет проникновение инфекции в исходно здоровые города. Одно только закрытие миграции при высоком R_0 малоэффективно. Многокритериальный поиск показал, что раннее выявление и низкая заразность выявленных случаев позволяют выполнить ограничение пика 30% и снизить смертность. Полный цикл воспроизводится из литературных источников командами Makefile.

Список литературы

1. Королькова А. В., Кулябов Д. С. Имитационное моделирование. Практикум. Москва: Российский университет дружбы народов имени Патриса Лумумбы, 2026.

2. Kermack W. O., McKendrick A. G. [A Contribution to the Mathematical Theory of Epidemics](#) // Proceedings of the Royal Society A. 1927. T. 115, № 772. С. 700–721.
3. Datseris G., Vahdati A. R., DuBois T. C. [Agents.jl: a performant and feature-full agent-based modeling software of minimal code complexity](#) // SIMULATION. 2022. Т. 98, № 11. С. 985–998.
4. Railsback S. F., Grimm V. Agent-Based and Individual-Based Modeling: A Practical Introduction. 2-е изд. Princeton University Press, 2019.
5. Datseris G. и др. [DrWatson: the perfect sidekick for your scientific inquiries](#) // Journal of Open Source Software. 2020. Т. 5, № 54. С. 2673.
6. Knuth D. E. [Literate Programming](#) // The Computer Journal. 1984. Т. 27, № 2. С. 97–111.